

Introduction to Web Design

Lecture Objectives

- By the end of this lecture, students should be able to:
1. Understand what web design is and why it is important
 2. Distinguish between websites and web applications
 3. Understand the basic components of a website
 4. Know the roles of HTML, CSS, and JavaScript
 5. Understand how websites work (client–server model)

Introduction to Web Design

1- What is Web Design?

Web design is the process of planning, creating, and maintaining websites. It includes:

1. Structure (HTML)
2. Appearance (CSS)
3. Behavior/interaction (JavaScript)
4. A good web design should be:
5. Easy to use (Usability)
6. Visually clear
7. Fast and responsive
8. Accessible on different devices (PC, mobile, tablet)

Introduction to Web Design

BAD



VS

GOOD



Introduction to Web Design

1. Your website is often the first impression of your brand, products, or services.
2. It is important to avoid common design mistakes that annoy users.
3. Good website design focuses on user experience.
4. A good design makes information clear, visually attractive, and easy to navigate.
5. Bad website design makes it hard for users to reach their goals.
6. Poor design also makes navigation and understanding content difficult.

Introduction to Web Design

2- Website vs Web Application

A- **Website:** Mostly provides information (e.g., news site, company site).

B- **Web Application:** Allows user interaction and processing (e.g., email, online banking).

3. Main Components of a Website

1. Frontend (Client-side)

1. What the user sees in the browser
2. Uses: HTML, CSS, JavaScript

2. Backend (Server-side)

1. Handles data, logic, authentication
2. Uses: PHP, Python, Node.js, Java, etc.

3. Database

1. Stores data (users, posts, products, etc.)

FRONT-END

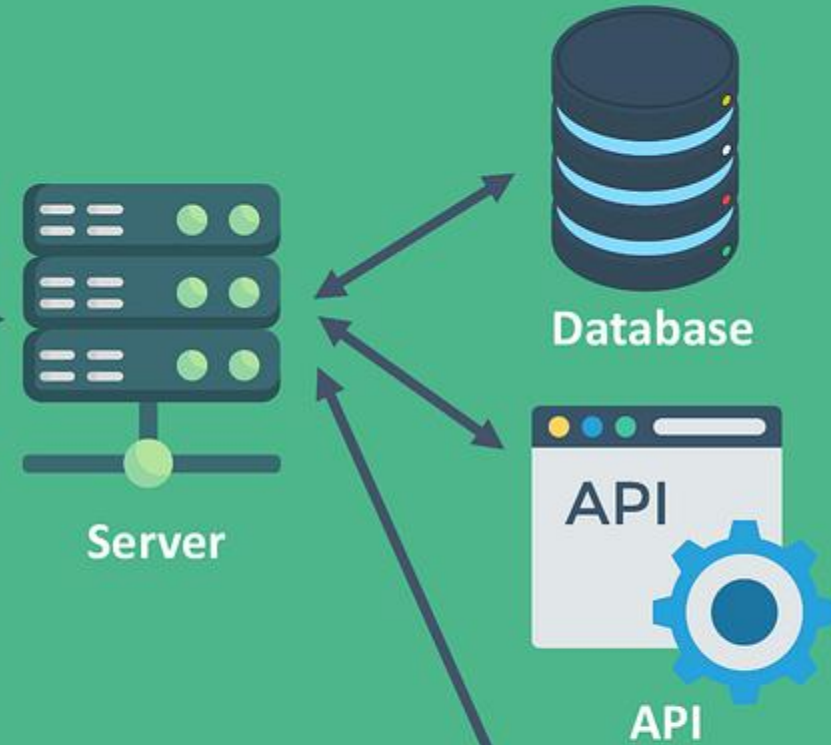


Web App

Mobile App



BACK-END



Server

Database

API



Introduction to Web Design

4- Core Web Technologies

1. HTML (**HyperText Markup Language**) → Structure of the page
2. CSS (**Cascading Style Sheets**) → Design and layout
3. JavaScript → **Interaction and behavior**

Example:

HTML = skeleton

CSS = clothes and style

JavaScript = movement and actions

Introduction to Web Design

7- introduction to Responsive Design

With more users browsing on mobile, having a non-responsive website is a major mistake. A responsive site adapts to all screen sizes and provides a smooth experience, while a non-responsive site forces users to zoom and scroll, causing frustration and higher bounce rates.

A- Responsive design means:

- 1- The website adapts to different screen sizes.
- 2- Works well on mobile, tablet, and desktop.

B- Non-responsive design: A non-responsive website requires users to pinch, zoom, and scroll to navigate; this can lead to significant frustration and often high bounce rates.

Introduction to Web Design

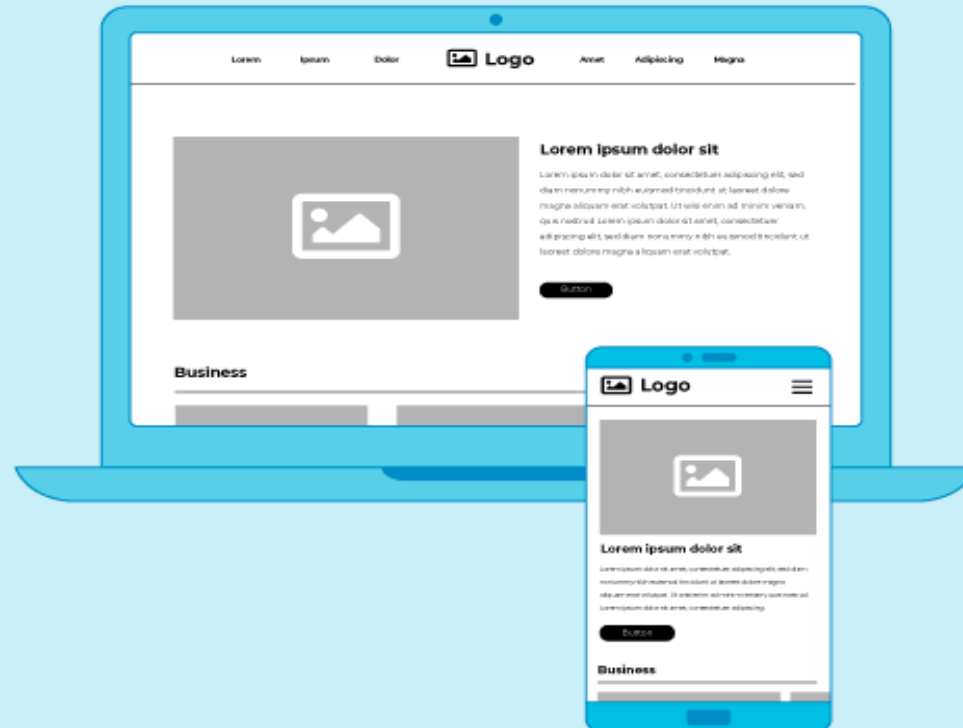
5- How the Web Works

1. User types a website address in the browser
2. Browser sends a request to the server
3. Server sends back HTML/CSS/JS files
4. Browser displays the page to the user

6- Tools for Web Design

1. Web browser (Chrome, Firefox, Edge)
2. Code editor (VS Code, Notepad++, etc.)
3. File system (folders for project files)

Introduction to Web Design



Introduction to Web Design

Notes about Responsive Non-responsive design

1- Poor Navigation: Easy-to-use navigation is essential if users are to find the information they're seeking and perform the tasks they want to achieve with your website. **Examples of poor navigation can include over-complicated menus with excessive numbers of sub-categories, or hidden menus with vague labels.**

2- Cluttered Layouts :Cluttered layouts overload users with **too much content and visual noise, making it hard to focus.** A clean, minimalist design with good organization, clear hierarchy, and enough white space helps highlight important information and guide users smoothly through the content.

3- Stylistic Inconsistency: Stylistic inconsistency makes a website look unprofessional and confusing. Using consistent fonts, colors, and design elements across all pages creates a unified, trustworthy look and improves user recognition and ease of use.

Introduction to Web Design



Introduction to Web Design

4- Slow Loading Speed: Users expect websites to load quickly and will leave if they have to wait too long. **Slow loading speeds increase bounce rates and harm search engine rankings, so improving performance with caching, optimized code, smaller files, and fewer ads is essential.**

5- Lack of clear CTAs: Without clear **CTAs (Call to Actions)**, users may not know what to do next, leading to missed conversions. **Strategically placed, easy-to-find CTAs with clear action words help guide users and improve user journeys.**

Example: Shop Now, Contact Us, Get Started.

Introduction to HTML

A web application consists of two main components:

- Front-end (Client-Side): This is what the user interacts with. It consists of HTML, CSS, and JavaScript. Runs in the user's web browser.
- Back-end (Server-Side) : Handles business logic, database operations, and user authentication. Uses server-side languages (e.g., Python, PHP, Node.js, Java). Interacts with databases (e.g., MySQL, MongoDB).

In this course, we will focus on the front-end of web applications, which includes:

HTML (HyperText Markup Language) → Structur

CSS (Cascading Style Sheets) → CSS is used to style and format the HTML elements.

JavaScript → JavaScript adds interactivity to a web page. It allows us to create dynamic content, animations, and real-time updates.

What is HTML? (HyperText Markup Language)

HTML was first introduced by Tim Berners-Lee in 1991 as a simple way to structure documents and link them together using hyperlinks.

Over the years, HTML has evolved through different versions, from HTML 2.0 to the current HTML5, introducing new elements and features to enhance web development.

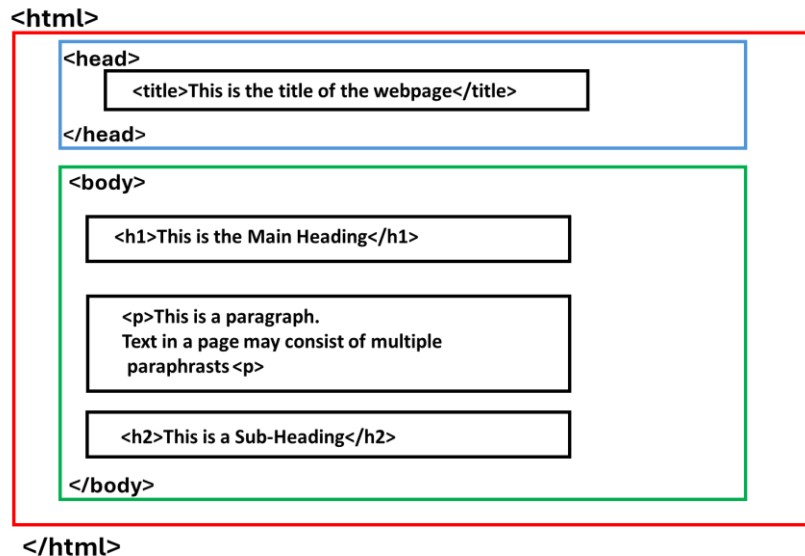
HTML is the foundation of any web page. It defines the structure of a webpage using tags.

Key Features of HTML Uses tags (e.g., <h1>, <p>, <a>,). Defines the content of a web page. HTML does not need a compiler (just an interpreter)

Example: Notepad + Internet Explorer (or other browsers like Google Chrome, Firefox, etc.)

HTML Structure

The basic structure of HTML page looks like this:



- **<html>**: The opening **<html>** tag indicates that anything between it and a closing **</html>** tag is HTML code.
- **<head>**: This tag represents the document's header which can keep other HTML tags like **<title>**, **<link>** etc.
- **<title>**: The **<title>** tag is used inside the **<head>** tag to mention the document title.
- **<body>**: tag indicates that anything between it and the closing **</body>** tag should be shown inside the main browser window.

Tags in HTML:

In HTML, there are opening tags and closing tags. The characters in the brackets indicate the tag's purpose. For example, in the tags above the **p** stands for paragraph. The closing tag has a forward slash after the **<** symbol.

<p>
Opening Tag

</p>
Closing Tag

An **Element** comprises the **opening tag** and the **closing tag** and any content that lies between them.

<p>Text</p>
Element

Similarly, an **Attribute** provide additional information about the contents of an element. They appear on the opening tag of the element and are made up of two parts: a name and a value, separated by an equals sign.

`<p style="text-align: center;">Text</p>`
Attribute

Basic Tags in HTML

Heading Tags: Any document starts with a heading. You can use different sizes for your headings. HTML also has six levels of headings, which use the elements `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, and `<h6>`. While displaying any heading, the browser adds one line before and one line after that heading.

<code><h1></code>	Hello	<code></h1></code>	Hello
<code><h2></code>	Hello	<code></h2></code>	Hello
<code><h3></code>	Hello	<code></h3></code>	Hello
<code><h4></code>	Hello	<code></h4></code>	Hello
<code><h5></code>	Hello	<code></h5></code>	Hello
<code><h6></code>	Hello	<code></h6></code>	Hello

Paragraph Tag `<p>`: The `<p>` tag offers a way to structure your text into different paragraphs. Each paragraph of text should go in between an opening `<p>` and a closing `</p>` tag.

<code><p></code>	This is a paragraph.	<code></p></code>	This is a paragraph.
------------------------	-----------------------------	-------------------------	----------------------

Line Break Tag `<br>`: Whenever you use the `<br>` element, anything following it starts from the next line. This tag is an example of an empty element, where you do not need opening and closing tags, as there is nothing to go in between them.

<code><p></code>	This is a paragraph without break line.	<code></p></code>
<code><p></code>	this is a paragraph	<code>
</code> with break line tag

Output:

This is a paragraph without break line.

this is a paragraph
with break line tag

Horizontal lines <hr>: are used to visually break-up sections of a document. The <hr> tag creates a line from the current position in the document to the right margin and breaks the line accordingly.

```
<p>This is a paragraph.</p>  
<hr>  
<p> This is another paragraph</p>
```

Output:

This is a paragraph.

This is another paragraph

Bold : Anything that appears within ... element, is displayed in bold

Strong : Anything that appears within ... element is displayed as important text.

Italic <i>: Anything that appears within <i>...</i> element is displayed in italicized

Emphasis : Anything that appears within this element is displayed as emphasized in italic.

```
<b>This text is bold.</b>  
<strong>This text is important.</strong>  
<i>This text is italicized.</i>  
<em>This text is emphasized.</em>
```

Output:

This text is bold. This text is important. *This text is italicized. This text is emphasized.*

Comment Tag: comments can be added to HTML source by using the following syntax: <!--
Write comments here -->

Superscript <sup>: The content of a ^{...} element is written in superscript; the font size used is the same size as the characters surrounding it but is displayed half a character's height above the other characters.

Subscript <sub>: The content of a _{...} element is written in subscript; the font size used is the same as the characters surrounding it, but is displayed half a character's height beneath the other characters.

```
H<sup>2</sup>O  <!-- Superscript example -->  
CO<sub>2</sub>  <!-- Subscript example -->
```

H²O
CO₂

Highlight <mark>: Anything that appears with-in <mark>...</mark> element, is displayed as marked with yellow ink.

```
<mark>This text is highlighted.</mark>
```

Output:

This text is highlighted.

Image : The tag is used to embed images in a webpage. The src attribute specifies the path to the image file. Other attributes control aspects such as alternative text, width, height.

```

```

Output:



HTML Grouping Using <div> and

- **<div> (Division Tag):** A block-level element used to group multiple elements together for styling or layout purposes.
- ** (Span Tag):** An inline element used to apply styles or scripts to a specific portion of text within a block.

HTML Lists

Lists help organize content in a structured manner. They enhance readability and improve the presentation of information. Lists are widely used in navigation menus, feature lists, instructions, and more.

Types of HTML Lists

- **Ordered Lists ():** Used when the order of items is important.
- **Unordered Lists ():** Used when order does not matter.
- **Definition Lists (<dl>):** Used for presenting terms and their descriptions.

Ordered Lists ()

- Ordered lists are used when the sequence of items is important.
- Each item is numbered or labeled sequentially (e.g., 1, 2, 3 or I, II, III).
- An ordered list starts with the tag. Each list item starts with the tag.
- The list items will be marked with numbers by default

```
<!-- Ordered List (ol) -->  
<ol>  
  <li>First item</li>  
  <li>Second item</li>  
  <li>Third item</li>  
</ol>
```

Output:

1. First item
2. Second item
3. Third item

Key Attributes of Ordered Lists

- **type:** Defines the numbering style (e.g., "1", "A", "a", "I", "i").
- **start:** Specifies the starting value of the numbering.
- **reversed:** Displays numbers in descending order.

```
<!-- Ordered List (ol) -->
<ol type="A">
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
```

A. First item
B. Second item
C. Third item

Unordered Lists ()

- Unordered lists are used when the order of items does not matter.
- An unordered list starts with the tag. Each list item starts with the tag.
- Items are displayed with bullet points instead of numbers.
- Different styles of bullets can be applied using CSS.

```
<!-- Unordered List (ul) -->
<ul>
  <li>Item one</li>
  <li>Item two</li>
  <li>Item three</li>
</ul>
```

Output:

- Item one
- Item two
- Item three

Definition Lists (<dl>)

- Definition lists are used for presenting terms and their corresponding descriptions.
- They consist of:
 - <dl> (Definition Lists): tag defines the description list
 - <dt> (Definition Term): Represents the term being defined.
 - <dd> (Definition Description): Contains the explanation or description of the term.
 -

```
<!-- Definition List (dl) -->
<dl>
  <dt>HTML</dt>
  <dd>A markup language for creating web pages.</dd>

  <dt>CSS</dt>
  <dd>A style sheet language used for describing the look of a document.</dd>
</dl>
```

Output:

HTML
A markup language for creating web pages.

CSS
A style sheet language used for describing the look of a document.

Nesting Lists

- Lists can be nested by placing one list inside another.
- This is commonly used for multi-level navigation menus or hierarchical content.

Code Sample

```
<html>
  <head>
    <title>HTML Examples</title>
  </head>
  <body>
    <!-- Headings -->
    <h1>Heading 1</h1>
    <h2>Heading 2</h2>
    <h3>Heading 3</h3>
    <h4>Heading 4</h4>
    <h5>Heading 5</h5>
    <h6>Heading 6</h6>
    <!-- Paragraph and Line Break -->
    <p>This is a paragraph of text.</p>
    <p>This is another paragraph.<br>This text appears on a new
line.</p>
    <!-- Horizontal Line -->
    <hr>
    <!-- Text Formatting -->
    <b>Bold Text</b><br>
    <strong>Strong (Important) Text</strong><br>
```

```
<i>Italicized Text</i><br>
<em>Emphasized Text</em><br>
<mark>Highlighted Text</mark><br>
<sup>Superscript Example (e.g., x2)</sup><br>
<sub>Subscript Example (e.g., H2O)</sub><br>
<!-- Comment -->
<!-- This is a comment in HTML and will not be displayed in the
browser -->
<!-- Image -->
<h3>Image Example</h3>


<!-- Grouping Elements -->
<div style="background-color: lightgray;">
  <h3>Div Example</h3>
  <p>This is inside a div container.</p>
</div>
<p>This is an example of <span style="color: red;">inline
text</span> using the <code>&lt;span&gt;</code> tag.</p>

<!-- Lists -->
<h3>Ordered List</h3>
<ol>
  <li>Item One</li>
  <li>Item Two</li>
  <li>Item Three</li>
</ol>
<h3>Unordered List</h3>
<ul>
  <li>Bullet Item 1</li>
  <li>Bullet Item 2</li>
  <li>Bullet Item 3</li>
</ul>
<h3>Definition List</h3>
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
```

```
    </dl>  
</body>  
</html>
```

HTML Tables

HTML tables are used to display **structured data** in rows and columns. They are commonly used for presenting **tabular data**, such as schedules, price lists, and reports.

A table in HTML is created using the `<table>` element. Inside a table, we use:

- `<tr>` for **table rows**
- `<td>` for **table data (cells)**
- `<th>` for **table headers** (bold & centered by default)

Basic Table Example

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>

  <tr>
    <td>Ibrahim</td>
    <td>6 </td>
  </tr>
</table>
```

Name	Age
Ibrahim	6

Table Borders and Spacing

By default, tables **don't have visible borders**. You can add borders using the border attribute.

Adding Borders

```

<table border="1">
  <tr>
    <th>Name</th>
    <th>Age</th>
    <th>Country</th>
  </tr>
  <tr>
    <td>Ahmed</td>
    <td>25</td>
    <td>Iraq</td>
  </tr>
  <tr>
    <td>Kareem</td>
    <td>30</td>
    <td>Jordan</td>
  </tr>
</table>

```

Name	Age	Country
Ahmed	25	Iraq
Kareem	30	Jordan

Adding a Table Caption

A **caption** describes the table's content.

```

<table border="1">
  <caption>Student Information</caption>
  <tr>
    <th>Name</th>
    <th>Age</th>
    <th>Grade</th>
  </tr>
  <tr>
    <td>Sama</td>
    <td>14</td>
    <td>A</td>
  </tr>
</table>

```

Student Information

Name	Age	Grade
Sama	14	A

Merging Cells (Colspan & Rowspan)

You can **merge cells** using:

- colspan (merges columns)
- rowspan (merges rows)

Example: Using colspan

```

<table border="1">
  <tr>
    <th colspan="2">Personal Details</th>
  </tr>
  <tr>
    <td>Name</td>
    <td>Mohammed</td>
  </tr>
</table>

```

Personal Details	
Name	Mohammed

Example: Using rowspan

```

<table border="1">
  <tr>
    <th rowspan="2">Day</th>
    <th>Morning</th>
    <th>Evening</th>
  </tr>
  <tr>
    <td>Exercise</td>
    <td>Study</td>
  </tr>
</table>

```

Day	Morning	Evening
	Exercise	Study

Nested Tables

A table inside another table is called a **nested table**.

```

<table border="1">
  <tr>
    <td>Main Table, Row 1</td>
    <td>
      <table border="1">
        <tr>
          <td>Nested Table, Row 1</td>
        </tr>
      </table>
    </td>
  </tr>
</table>

```

HTML Styles

The HTML **style** attribute is used to add styles to an element, such as color, font, size, and more.

```
<!DOCTYPE html>
<html>
<body>

<p>I am normal</p>
<p style="color:red;">I am red</p>
<p style="color:blue;">I am blue</p>
<p style="font-size:50px;">I am big</p>

</body>
</html>
```

I am normal

I am red

I am blue

I am big

The HTML Style Attribute

Setting the style of an HTML element, can be done with the **style** attribute.

The HTML **style** attribute has the following syntax:

```
<tagname style="property:value;">
```

The **property** is a CSS property. The **value** is a CSS value.

Background Color

The CSS **background-color** property defines the background color for an HTML element.

```
<!DOCTYPE html>
<html>
<body style="background-color:powderblue;">

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

This is a heading

This is a paragraph.

Example

Set background color for two different elements:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="background-color:powderblue;">This is a
heading</h1>
<p style="background-color:tomato;">This is a paragraph.
</p>

</body>
</html>
```

This is a heading

This is a paragraph.

Text Color

The CSS **color** property defines the text color for an HTML element:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>

</body>
</html>
```

This is a heading

This is a paragraph.

Fonts

The CSS **font-family** property defines the font to be used for an HTML element:

Example

```
<!DOCTYPE html>
<html>
<body>

<h1 style="font-family:verdana;">This is a heading</h1>
<p style="font-family:courier;">This is a paragraph.</p>

</body>
</html>
```

This is a heading

This is a paragraph.

Text Size

The CSS **font-size** property defines the text size for an HTML element:

Example

```
<!DOCTYPE html>
<html>
<body>

<h1 style="font-size:300%;">This is a heading</h1>
<p style="font-size:160%;">This is a paragraph.</p>

</body>
</html>
```

This is a heading

This is a paragraph.

Text Alignment

The CSS **text-align** property defines the horizontal text alignment for an HTML element:

Example

```
<!DOCTYPE html>
<html>
<body>

<h1 style="text-align:center;">Centered Heading</h1>
<p style="text-align:center;">Centered paragraph.</p>

</body>
</html>
```

Centered Heading

Centered paragraph.

HTML Comment Tag

You can add comments to your HTML source by using the following syntax:

```
<!DOCTYPE html>
<html>
<body>

<!-- This is a comment -->
<p>This is a paragraph.</p>
<!-- Comments are not displayed in the browser -->

</body>
</html>
```

This is a paragraph.

Hide Content

Comments can be used to hide content.

This can be helpful if you hide content temporarily:

Example

```
<!DOCTYPE html>
<html>
<body>

<p>This is a paragraph.</p>

<!-- <p>This is another paragraph </p> -->

<p>This is a paragraph too.</p>

</body>
</html>
```

This is a paragraph.

This is a paragraph too.

→You can also hide more than one line. Everything between the **<!--** and the **-->** will be hidden from the display.

Example:

```
<!DOCTYPE html>
<html>
<body>

<p>This is a paragraph.</p>
<!--
<p>Look at this cool image:</p>

-->
<p>This is a paragraph too.</p>

</body>
</html>
```

This is a paragraph.

This is a paragraph too.

Comments are also great for debugging HTML, because you can comment out HTML lines of code, one at a time, to search for errors.

Hide Inline Content

Comments can be used to hide parts in the middle of the HTML code.

```
<!DOCTYPE html>
<html>
<body>

<p>This <!-- great text --> is a paragraph.</p>

</body>
</html>
```

This is a paragraph.

HTML colors are specified with predefined color names, or with RGB, HEX, HSL, RGBA, or HSLA values.

Color Names

In HTML, a color can be specified by using a color name:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="background-color:Tomato;">Tomato</h1>
<h1 style="background-color:Orange;">Orange</h1>
<h1 style="background-color:DodgerBlue;">DodgerBlue</h1>
<h1 style="background-
color:MediumSeaGreen;">MediumSeaGreen</h1>
<h1 style="background-color:Gray;">Gray</h1>
<h1 style="background-color:SlateBlue;">SlateBlue</h1>
<h1 style="background-color:Violet;">Violet</h1>
<h1 style="background-color:LightGray;">LightGray</h1>

</body>
</html>
```

Tomato

Orange

DodgerBlue

MediumSeaGreen

Gray

SlateBlue

Violet

LightGray

Background Color

You can set the background color for HTML elements:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="background-color:DodgerBlue;">Hello World</h1>

<p style="background-color:Tomato;">
Lorem ipsum dolor sit amet, consectetur adipiscing elit,
sed diam nonummy nibh euismod tincidunt ut laoreet dolore
magna aliquam erat volutpat.
Ut wisi enim ad minim veniam, quis nostrud exerci tation
ullamcorper suscipit lobortis nisl ut aliquip ex ea
commodo consequat.
</p>

</body>
</html>
```

Hello World

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.

Text Color

You can set the color of text:

```
<!DOCTYPE html>
<html>
<body>

<h3 style="color:Tomato;">Hello World</h3>

<p style="color:DodgerBlue;">Lorem ipsum dolor sit amet,
consectetur adipiscing elit, sed diam nonummy nibh
euismod tincidunt ut laoreet dolore magna aliquam erat
volutpat.</p>

<p style="color:MediumSeaGreen;">Ut wisi enim ad minim
veniam, quis nostrud exerci tation ullamcorper suscipit
lobortis nisl ut aliquip ex ea commodo consequat.</p>

</body>
</html>
```

Hello World

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.

Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.

Border Color

You can set the color of borders:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="border: 2px solid Tomato;">Hello World</h1>

<h1 style="border: 2px solid DodgerBlue;">Hello
World</h1>

<h1 style="border: 2px solid Violet;">Hello World</h1>

</body>
</html>
```

Hello World

Hello World

Hello World

Color Values

In HTML, colors can also be specified using RGB values, HEX values, HSL values, RGBA values, and HSLA values.

The following three <div> elements have their background color set with RGB, HEX, and HSL values:

```
<!DOCTYPE html>
<html>
<body>

<p>Same as color name "Tomato":</p>

<h1 style="background-color:rgb(255, 99, 71);">rgb(255,
99, 71)</h1>
<h1 style="background-color:#ff6347;">#ff6347</h1>
<h1 style="background-color:hsl(9, 100%, 64%);">hsl(9,
100%, 64%)</h1>

<p>Same as color name "Tomato", but 50% transparent:</p>
<h1 style="background-color:rgba(255, 99, 71,
0.5);">rgba(255, 99, 71, 0.5)</h1>
<h1 style="background-color:hsla(9, 100%, 64%,
0.5);">hsla(9, 100%, 64%, 0.5)</h1>

<p>In addition to the predefined color names, colors can
be specified using RGB, HEX, HSL, or even transparent
colors using RGBA or HSLA color values.</p>

</body>
</html>
```

Same as color name "Tomato":

rgb(255, 99, 71)

#ff6347

hsl(9, 100%, 64%)

Same as color name "Tomato", but 50% transparent:

rgba(255, 99, 71, 0.5)

hsla(9, 100%, 64%, 0.5)

In addition to the predefined color names, colors can be specified using RGB, HEX, HSL, or even transparent colors using RGBA or HSLA color values.

What is CSS?

Cascading Style Sheets (CSS) is used to format the layout of a webpage.

With CSS, you can control the color, font, the size of text, the spacing between elements, how elements are positioned and laid out, what background images or background colors are to be used, different displays for different devices and screen sizes, and much more!

Using CSS

CSS can be added to HTML documents in 3 ways:

- **Inline** - by using the **style** attribute inside HTML elements
- **Internal** - by using a **<style>** element in the **<head>** section
- **External** - by using a **<link>** element to link to an external CSS file

The most common way to add CSS, is to keep the styles in external CSS files. However, in this tutorial we will use inline and internal styles, because this is easier to demonstrate, and easier for you to try it yourself.

Inline CSS

An inline CSS is used to apply a unique style to a single HTML element.

An inline CSS uses the `style` attribute of an HTML element.

The following example sets the text color of the `<h1>` element to blue, and the text color of the `<p>` element to red:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;">A Blue Heading</h1>

<p style="color:red;">A red paragraph.</p>

</body>
</html>
```

A Blue Heading

A red paragraph.

Internal CSS

An internal CSS is used to define a style for a single HTML page.

An internal CSS is defined in the `<head>` section of an HTML page, within a `<style>` element.

The following example sets the text color of ALL the `<h1>` elements (on that page) to blue, and the text color of ALL the `<p>` elements to red. In addition, the page will be displayed with a "powderblue" background color:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {background-color: powderblue;}
h1   {color: blue;}
p    {color: red;}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

This is a heading

This is a paragraph.

External CSS

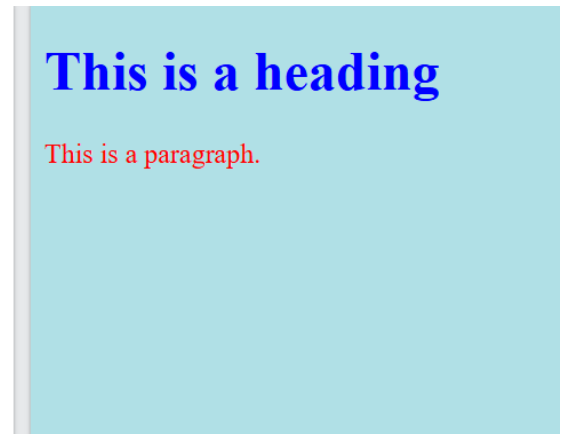
An external style sheet is used to define the style for many HTML pages.

To use an external style sheet, add a link to it in the `<head>` section of each HTML page:

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```



The external style sheet can be written in any text editor. The file must not contain any HTML code, and must be saved with a .css extension.

Here is what the "styles.css" file looks like:

"styles.css":

```
body {
  background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

```
<!DOCTYPE html>
<html>
<head>
<style>
h1 {
  color: blue;
  font-family: verdana;
  font-size: 300%;
}
p {
  color: red;
  font-family: courier;
  font-size: 160%;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

This is a heading

This is a paragraph.

CSS Border

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  border: 2px solid powderblue;
}
</style>
</head>
<body>

<h1>This is a heading</h1>

<p>This is a paragraph.</p>
<p>This is a paragraph.</p>
<p>This is a paragraph.</p>

</body>
</html>
```

This is a heading

This is a paragraph.

This is a paragraph.

This is a paragraph.

CSS Padding

The CSS **padding** property defines a padding (space) between the text and the border.

```

<!DOCTYPE html>
<html>
<head>
<style>
p {
  border: 2px solid powderblue;
  padding: 30px;
}
</style>
</head>
<body>

<h1>This is a heading</h1>

<p>This is a paragraph.</p>
<p>This is a paragraph.</p>
<p>This is a paragraph.</p>

</body>
</html>

```

This is a heading

This is a paragraph.

This is a paragraph.

This is a paragraph.

CSS Margin

The CSS **margin** property defines a margin (space) outside the border.

```

<!DOCTYPE html>
<html>
<head>
<style>
p {
  border: 2px solid powderblue;
  margin: 50px;
}
</style>
</head>
<body>

<h1>This is a heading</h1>

<p>This is a paragraph.</p>
<p>This is a paragraph.</p>
<p>This is a paragraph.</p>

</body>
</html>

```

This is a heading

This is a paragraph.

This is a paragraph.

This is a paragraph.

Link to External CSS

External style sheets can be referenced with a full URL or with a path relative to the current web page.

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="/html/styles.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

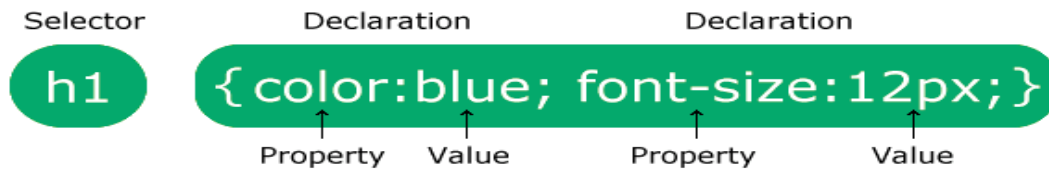
</body>
</html>
```

This is a heading

This is a paragraph.

What is CSS?

- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen.
- CSS saves a lot of work. It can control the layout of multiple web pages all at once
- External stylesheets are stored in CSS files



The selector points to the HTML element you want to style.

The declaration block contains one or more declarations separated by semicolons.

Each declaration includes a CSS property name and a value, separated by a colon.

Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.

Types of selectors:

Element selector	P	All <p> elements
ID selector	#abc	element with id="abc"
Class selector	.abc	elements with class="abc"

```
<html>
<head>
<title>My first web</title>
<style>
  body{background-color: LightGray;}
  #title{color: Blue;}
  .hw{ color:green}
</style>
</head>
<body>
  <h1 id="title">Homework</h1>
  <em class="hw">HW0</em> is due Friday.<br/>
  <em class="hw">HW1</em> goes out Monday.<br/>
  <em>All homework due at 11:59pm.</em>
</body>
</html>
```

Homework

HW0 is due Friday.
HW1 goes out Monday.
All homework due at 11:59pm.

Three Ways to Insert CSS

There are three ways of inserting a style sheet:

- External CSS
 - Internal CSS
 - Inline CSS
-

Inline CSS

An inline style may be used to apply a unique style for a single element.

To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.

```
<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>
```

Internal CSS

An internal style sheet may be used if one single HTML page has a unique style.

The internal style is defined inside the <style> element, inside the head section.

```
<head>
<style>
body {
    background-color: maroon;
}

h1 {
    color: white;
    margin-left: 40px;
}
</style>
</head>
```

External CSS

With an external style sheet, you can change the look of an entire website by changing just one file! Each HTML page must include a reference to the external style sheet file inside the <link> element, inside the head section. The <link> element can be used in an HTML document to tell the browser where to find the CSS file used to style the page. It is an empty element (meaning it does not need a closing tag), and it

lives inside the <head> element.

```
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
```

It should use three attributes:

- **href:** This specifies the path to the CSS file
- **rel:** This specifies the relationship between the HTML page and the file it is linked to. The value should be **stylesheet** when linking to a CSS file.

Note: The **link element** can be used for other purposes, for example it can be used to add an icon to your web page:

```
<link rel="icon" type="image/x-icon" href="icon.png">
```

Output:



CSS background properties

The CSS background properties are used to add background effects for elements. Such as:

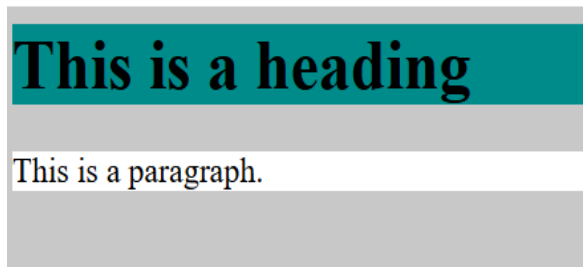
- background-color
- background-image
- background-repeat
- background-attachment
- background-position

background-color:

```
<head>
<style>
body {
    background-color: rgb(200,200,200);
h1 {
    background-color: DarkCyan;}
p {
    background-color: white;}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
```



This is a heading

This is a paragraph.

background-image

```
body {  
background-image: url("paper-texture.gif");  
}
```



CSS Borders

The border-style property specifies what kind of border to display.

```
<head>  
  <style>  
    div{border: 4px dotted black;}  
  </style>  
</head>  
<body>  
  <div>dotted</div>  
</body>
```

dotted - Defines a dotted border

dashed - Defines a dashed border

solid - Defines a solid border

double - Defines a double border

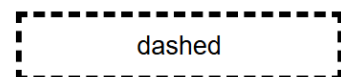
none - Defines no border

hidden - Defines a hidden border

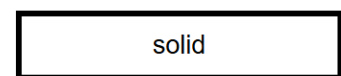
groove, ridge, inset, outset - 3D effects, The effect depends on the border-color value



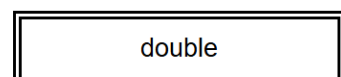
dotted



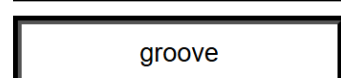
dashed



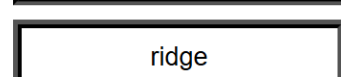
solid



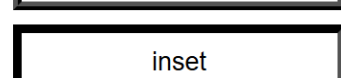
double



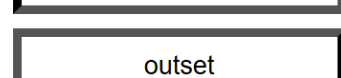
groove



ridge



inset



outset

none

hidden

The border-style property can have from one to four values (for the top border, right border, bottom border, and the left border).

```

<head>
  <style>
    p{border-style: dotted dashed solid double;}
  </style>
</head>
<body>
  <p> Mixed border</p>
</body>

```

Output:

Mixed border

CSS Border Width

The border-width property specifies the width of the four borders.

The width can be set as:

- specific size (in px, pt, cm, em, etc) or by using one of the three
- pre-defined values: thin, medium, or thick

CSS Border Color

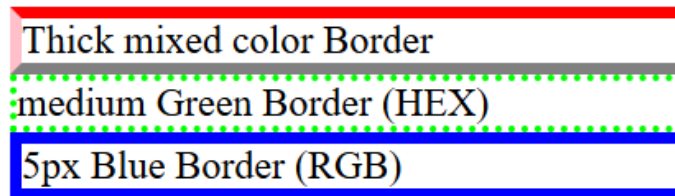
The border-color property is used to set the color of the four borders. The color can be set by:

- name - specify a color name, like "red"
- HEX - specify a HEX value, like "#ff0000"
- RGB - specify a RGB value, like "rgb(255,0,0)"

```

<head>
  <style>
    .border-mix {
      border-color: red green gray pink;
      Border-style: solid;
      border-width: thick ;
    }
    .border-hex {
      border-color: #00ff00;
      Border-style: dotted;
      border-width: medium;
    }
    .border-rgb {
      border-color: rgb(0, 0, 255);
      Border-style: solid;
      Border-width: 5px;
    }
  </style>
</head>
<body>
  <div class="border-mix">Thick mixed color Border</div>
  <div class="border-hex">medium Green Border (HEX)</div>
  <div class="border-rgb">5px Blue Border (RGB)</div>
</body>

```



Another border style property to make rounded corners is **border-radius**, for example:
border-radius :10px;

CSS Margins

The CSS margin properties are used to create space around elements, outside of any defined borders. With CSS, you have full control over the margins. There are properties for setting the margin for each side of an element (top, right, bottom, and left).

Setting Margins

You can set margins for each side of an element using:

- margin-top
- margin-right
- margin-bottom
- margin-left

```
<style>
  .margin-example {
    margin-top: 70px;
    margin-right: 150px;
    margin-bottom: 40px;
    margin-left: 50px;
    border: 2px solid black;}
</style>
</head>
<body>
  <div class="margin-example">This box has margins</div>
</body>
```

This box has margins

CSS Padding

The CSS padding properties are used to generate space around an element's content, inside of any defined borders. With CSS, you have full control over the padding. There are properties for setting the padding for each side of an element (top, right, bottom, and left)

Padding - Individual Sides

CSS has properties for specifying the padding for each side of an element:

- padding-top
- padding-right
- padding-bottom
- padding-left

```
<style>
.padding-example {
padding-top: 10px;
padding-right: 20px;
padding-bottom: 30px;
padding-left: 40px;
border: 2px solid black;}
</style>
</head>
<body>
<div class="padding-example">This box has padding</div>
</body>
```



Or:

```
<head>
<style>
.padding-example {padding: 10px 20px 30px 40px; /* Top Right Bottom Left */}
</style>
</head>
<body>
<div class="padding-example">This box has padding</div>
</body>
```

Text Color

The color property is used to set the color of the text. The color is specified by:

- a color name - like "red"
- a HEX value - like "#ff0000"
- an RGB value - like "rgb(255,0,0)"

HTML Lists and CSS List Properties

The CSS list properties allow you to:

- Set different list item markers for ordered lists
- Set different list item markers for unordered lists
- Set an image as the list item marker
- Add background colors to lists and list items

```
<head>
<style>
  ul { list-style-image: url('icon.png'); }
  ol { list-style-type: upper-roman;
        background-color: lightblue;}
</style>
</head>
<body>
  <ul>
    <li>Apple</li>
    <li>Banana</li>
  </ul>
  <ol>
    <li>Step 1</li>
    <li>Step 2</li>
  </ol>
</body>
```

✓ Apple
✓ Banana

I. Step 1
II. Step 2

Block vs. Inline Elements in HTML

In HTML, elements are categorized into **block-level** and **inline** elements based on how they are displayed on the page.

1. Block Elements

A **block-level element** always **starts on a new line** and takes up the **full width** available.

- Always starts on a **new line**
- Takes up the **full width** available by default
- Can have **width, height, margin, and padding**

```
<p>This is a paragraph.</p> <!-- Block element -->
<div>This is a div.</div> <!-- Block element -->
<h1>This is a heading</h1> <!-- Block element -->
```

2. Inline Elements

An **inline element** stays **on the same line** as surrounding content and only takes up as much width as necessary.

- Does **not** start on a new line
- Only takes up as much width as necessary
- **Cannot** have width and height applied

```
<span>This is a span</span> <!-- Inline element -->
<a href="#">This is a link</a> <!-- Inline element -->
<strong>This is bold text</strong> <!-- Inline element -->
```

The **display property** in CSS determines how an HTML element behaves in the document layout. It controls how elements are displayed on a webpage.

display: block Allows an element with display: block to behave as a **block-level element**.

display: inline Allows an element with display: inline to behave as an **inline element**

inline-block is a **hybrid** between **block** and **inline** elements.

It allows an element to **stay inline**, but also **set width, height, and margins** like a block element.

CSS Layout - Align

To just center the text inside an element, use text-align: center;

CSS Layout - float and clear

The CSS float property specifies how an element should float.

The CSS clear property specifies what elements can float beside the cleared element and on which side.

CSS Tables

The border-collapse property sets whether the table borders should be collapsed into a single border:

```
<style>
table {
border-collapse: collapse;
width: 50%;
}

th, td {
border: 1px solid black;
padding: 8px;
text-align: left;
}
</style>
```

Name	Age
John	25

Lecture 5: CSS Flexbox, Class Attribute, Forms, and External Stylesheets

Introduction

In this lecture, we will cover essential CSS and HTML concepts, including the flex property, class attributes, form elements, and external stylesheets. These concepts are fundamental for designing responsive and well-structured web pages.

1. CSS Flexbox

What is Flexbox?

Flexbox is a CSS layout model that allows items within a container to be arranged efficiently, even when their sizes are dynamic. It is designed for one-dimensional layouts, meaning it aligns items either in a row (horizontally) or in a column (vertically).

The `display: flex;` Property

To enable Flexbox, you need to set the `display` property of a container to `flex`.

```
.container {  
  display: flex;  
}
```

This makes all direct child elements of `.container` become flex items.

Flexbox Properties

1. `flex-direction`

Determines the direction of the flex items.

- `row` (default): Items are placed left to right.
- `row-reverse`: Items are placed right to left.

- column: Items are stacked vertically.
- column-reverse: Items are stacked in reverse order.

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

2. justify-content

Aligns items along the main axis.

- flex-start: Items align to the start.
- flex-end: Items align to the end.
- center: Items are centered.
- space-between: Items are evenly spaced, with the first at the start and the last at the end.
- space-around: Items have equal spacing around them.
- space-evenly: Items have equal spacing between and around them.

Example:

```
.container {  
  display: flex;  
  justify-content: space-between;  
}
```

3. align-items

Aligns items along the cross-axis (perpendicular to flex-direction).

- flex-start: Items align at the start.
- flex-end: Items align at the end.
- center: Items are centered.

- stretch: Items stretch to fill the container.
- baseline: Items align according to their text baseline.

Example:

```
.container {  
  display: flex;  
  align-items: center;  
}
```

Examples

Example 1: Simple Flexbox Layout

```
<html>  
<head>  
<style>  
  .container {  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    height: 300px;  
    border: 1px solid black;  
  }  
  .box {  
    width: 100px;  
    height: 100px;  
    margin: 10px;  
    background-color: lightblue;  
  }  
</style>  
</head>  
<body>  
<div class="container">  
  <div class="box">1</div>
```

```
<div class="box">2</div>
<div class="box">3</div>
</div>
</body>
</html>
```

Result: Three equally spaced boxes, centered both horizontally and vertically.

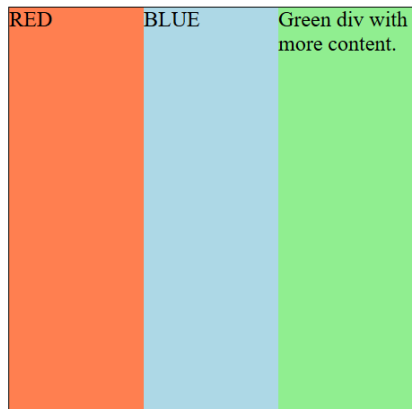


Example 2:

```
<html>
<head>
<style>
#main {
width: 300px;
height: 300px;
border: 1px solid black;
display: flex;
}
#main div {
flex: 1;
}
</style>
</head>
<body>
<h1>The flex Property</h1>
<div id="main">
<div style="background-color:coral;">RED</div>
```

```
<div style="background-color:lightblue;">BLUE</div>
<div style="background-color:lightgreen;">Green div with more
content.</div>
</div>
</body> </html>
```

The flex Property



Example 3: Responsive Navbar with Flexbox

```
<html>
<head>
<style>
.navbar {
  display: flex;
  justify-content: space-between;
  background-color: #333;
  padding: 10px;
}
.navbar a {
  color: white;
  text-decoration: none;
  padding: 10px;
}
</style>
</head>
<body>
<div class="navbar">
```

```
<a href="#">Home</a>
<a href="#">About</a>
<a href="#">Contact</a>
</div>
</body>
</html>
```

Result: A responsive navigation bar with evenly spaced links.

2. HTML Class Attribute

The class attribute allows multiple HTML elements to share the same styling by assigning a common name. This is useful for maintaining consistency and reducing redundant code.

Example 5: Using class for Styling

```
<html>
<head>
<style>
.city {
  background-color: tomato;
  color: white;
  border: 2px solid black;
  margin: 20px;
  padding: 20px;
}
</style>
</head>
<body>
<div class="city">
  <h2>London</h2>
  <p>London is the capital of England.</p>
</div>
<div class="city">
  <h2>Paris</h2>
```

```
<p>Paris is the capital of France.</p>
</div>
<div class="city">
  <h2>Tokyo</h2>
  <p>Tokyo is the capital of Japan.</p>
</div>
</body>
</html>
```



Key Points:

- The .city class applies the same styles to all <div> elements that have the city class.
- Classes can also be used in JavaScript to manipulate elements dynamically.

3. HTML Forms

Forms are an essential part of web applications as they allow users to input and submit data.

Form Attributes

1. **action:** Specifies the URL where form data should be sent.

2. **method:** Defines how form data is submitted (GET or POST).
3. **target:** Determines where the response is displayed:
 - `_blank`: Opens in a new tab.
 - `_self`: Opens in the same window (default).
 - `_parent`: Opens in the parent frame.
 - `_top`: Opens in the full body of the window.

The HTML <form> Elements:

The HTML <form> element can contain one or more of the following form elements:

- <input>
- <label>
- <select> <option>
- <textarea>
- <button>
- <Submit>
- <Reset>

The <input> tag specifies an input field where the user can enter data. The <input> element is the most important form element. The <input> element can be displayed in several ways, depending on the type attribute. The different input types are as follows:

- <input type="button">
- <input type="checkbox">
- <input type="color">
- <input type="date">
- <input type="password">
- <input type="radio">
- <input type="reset">
- <input type="submit">

The Method Attribute

The method attribute specifies the HTTP method to be used when submitting the form data.

The form-data can be sent as URL variables (with method="get") or as HTTP post transaction (with method="post").

The default HTTP method when submitting form data is GET.

Notes on GET:

- Appends the form data to the URL, in name/value pairs
- NEVER use GET to send sensitive data! (the submitted form data is visible in the URL!)
- The length of a URL is limited (2048 characters)
- Useful for form submissions where a user wants to bookmark the result
- GET is good for non-secure data, like query strings in Google

Notes on POST:

- Appends the form data inside the body of the HTTP request (the submitted form data is not shown in the URL)
- POST has no size limitations, and can be used to send large amounts of data.
- Form submissions with POST cannot be bookmarked

Example:

```
<form action="submit.php" method="get">
  <input type="text" name="name" value="John">
  <input type="submit">
</form>
```

The URL after submission might be:

<https://example.com/submit.php?name=John>

```
<form action="submit.php" method="post">
  <input type="text" name="name" value="John">
  <input type="submit">
</form>
```

The data is sent in the request body, so the URL remains without any query parameters. <https://example.com/submit.php>

Example 6: Creating a Form

```
<html>
<head> </head>
<body>
<form action="Lecture2.html" method="get">
<label for="user">user name</label>
<Input type="text" name="user" id="user1"
placeholder="Enter your name"><br>
<label>Password</label>
<Input type="Password" name="pass"><br>
<input type="checkbox" id="ch"><label for="ch">Remember
me</label> <br>
<input type="Radio" id="m" name="Gen">
<label for="m">Male</label>
<input type="Radio" id="f" name="Gen">
<label for="m">Female</label><br>
<input type="Radio" id="m1" name="mm">
<label for="m">Single</label>
<input type="Radio" id="f1" name="mm">
<label for="m">Married</label>
<Input type="color">
<input type="date">
<select name="a" id="aa">
<option>Baghdad</option>
<option>London</option>
<option>Paris</option>
</select> <br>
```

```
<Input type="submit">
<Input type="Reset" >
</body>
</html>
```

Result:

user name

Password

☐ Remember me

☐ Male ☐ Female

☐ Single ☐ Married

4. CSS Forms Styling

CSS can be used to improve the appearance of forms.

Example 7: Styling Input Fields

```
input[type=text], select, textarea {
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 4px;
    resize: vertical;
}

input[type=submit] {
    background-color: #04AA6D;
    color: white;
    padding: 12px 20px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}
```

```

    float: right;
}
input[type=submit]:hover {
    background-color: #45a049;
}
.container {
    background-color: #f2f2f2;
    padding: 20px;
    border-radius: 5px;
}

```

Responsive Form Example

```

<div class="container">
  <form action="/action_page1.html">
    <div class="row">
      <label for="fname">First Name:</label>
      <input type="text" id="fname" name="firstname"
placeholder="Your name..">
    </div>
    <div class="row">
      <label for="lname">Last Name:</label>
      <input type="text" id="lname" name="lastname"
placeholder="Your last name..">
    </div>
    <div class="row">
      <label for="country">Country:</label>
      <select id="country" name="country">
        <option value="australia">Australia</option>
        <option value="canada">Canada</option>
        <option value="usa">USA</option>
      </select>
    </div>
    <input type="submit" value="Submit">
  </form>
</div>

```

Result without style

First Name:

Last Name:

Country:

Result with Style

First Name:

Last Name:

Country:

5. External Stylesheets

Instead of writing CSS in the `<style>` tag, we can use an external CSS file for better code organization.

Example 8: `style.css`

```
body {  
    color: blue;  
    background-color: coral;  
    background-image: url("p1.jpg");  
    background-repeat: repeat;  
    background-position: center;  
    background-size: 50%;  
}  
  
p {  
    color: rgb(50,100,150);  
    border: 10px solid brown;  
}  
  
h2 {  
    text-align: right;
```

```
    font-size: 50px;  
}
```

Linking the Stylesheet in HTML:

```
<link rel="stylesheet" href="style.css">
```

JavaScript

How to - Become a Front-End Developer

A Front-End Developer is someone who creates websites and web applications. The Front-End Developer creates things that the user sees. It is a popular job, and everyone can become a Front-End Developer.

What does a Front-End Developer do?

The main responsibility of a Front-End Developer is the **User interface**. Simply put, create things that the user sees.

The difference between Front-End and Back-End is that Front-End refers to how a web page looks, while back-end refers to how it works. You also think of Front-End as **client-side** and Back-End as **server-side**.

JavaScript is the world's most popular programming language. JavaScript is the programming language of the Web.

What is JavaScript?

JavaScript is the programming language of the web. It can update and change both HTML and CSS. It can calculate, manipulate and validate data.

Why Study JavaScript?

JavaScript is one of the 3 languages all web developers must learn:

1. **HTML to Create the structure with HTML.** The first thing you have to learn is HTML, which is the standard mark-up language for creating web pages.
2. **CSS to Style with CSS.** The next step is to learn CSS, to set the layout of your web page with beautiful colours, fonts, and much more.
3. **JavaScript** to program the behaviour of web pages and make it interactive with JavaScript. After studying HTML and CSS, you should learn JavaScript to create dynamic and interactive web pages for your users.

JavaScript Can Change HTML Content

One of many JavaScript HTML methods is `getElementById()`.

The example below "finds" an HTML element (with `id="par"`), and changes the element content (`innerHTML`) to "Hello JavaScript":

```
document.getElementById("demo").innerHTML = "Hello JavaScript";
```

```
<html>
```

```
<html>
```

```
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p id="par">JavaScript can change HTML content.</p>
```

```
<button type="button" onclick='document.getElementById("par").innerHTML =  
"Hello JavaScript!'">Click Me!</button>
```

```
</body>
```

```
</html>
```

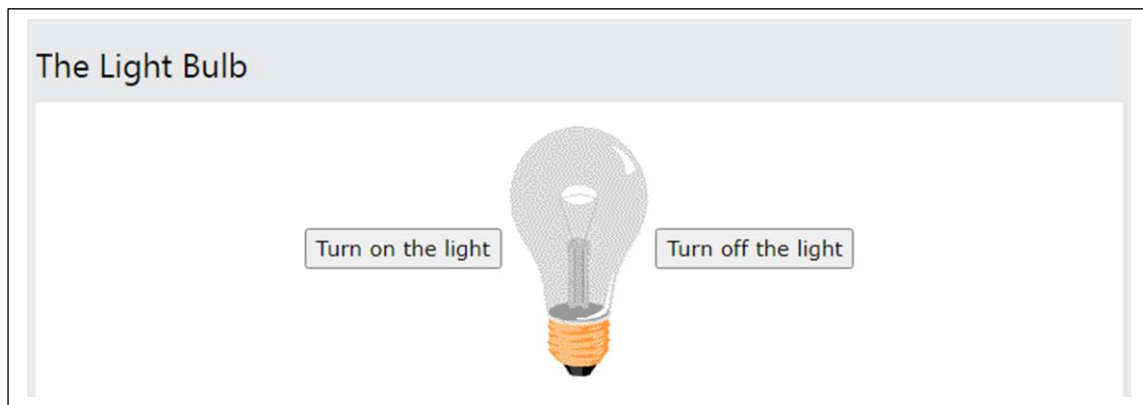
What Can JavaScript Do?

JavaScript can change HTML content.

Click Me!

JavaScript Can Change HTML Attribute Values

In this example JavaScript changes the value of the **src** (source) attribute of an **** tag:



```
<html>
```

```
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p>JavaScript can change HTML attribute values.</p>
```

```
<p>In this case JavaScript changes the value of the src (source) attribute of an image.</p>
```

```
<button onclick="document.getElementById('myImage').src='b2.jpg'">Turn on the light</button>
```

```

```

```
<button onclick="document.getElementById('myImage').src='b1.jpg'">Turn off the light</button>
```

```
</body>
```

```
<html>

<head>

<script>

function myFunction() {

    document.getElementById("demo").innerHTML = "Paragraph changed.";

}

</script>

</head>

<body>

<h2>Demo JavaScript in Head</h2>

<p id="demo">A Paragraph.</p>

<button type="button" onclick="myFunction()">Try it</button>

</body>

</html>
```

Window Console Object

The Console Object

The **console object** provides access to the browser's debugging console.

The **console object** is a property of the **window object**.

The **console object** is accessed with:

`window.console` or just `console`

Example 1

```
<html>

<body>

<h1>The Console Object</h1>

<h2>The error() Method</h2>

<p>Remember to open the console (Press F12) before you click "Run".</p>

<script>

console.error("You made a mistake");

</script>

</body>

</html>
```

Console Object Methods

Method	Description
assert()	Writes an error message to the console if a assertion is false
clear()	Clears the console
count()	Logs the number of times that this particular call to count() has been called
error()	Outputs an error message to the console
group()	Creates a new inline group in the console. This indents following console messages by an additional level, until console.groupEnd() is called
groupCollapsed()	Creates a new inline group in the console. However, the new group is created collapsed. The user will need to use the disclosure button to expand it
groupEnd()	Exits the current inline group in the console
info()	Outputs an informational message to the console
log()	Outputs a message to the console

table() Displays tabular data as a table

time() Starts a timer (can track how long an operation takes)

timeEnd() Stops a timer that was previously started by console.time()

JavaScript Error message

```
<html>
```

```
<body>
```

```
<h1>JavaScript Errors</h1>
```

```
<p>In this example we have written "alert" as "adddalert" to deliberately produce an error.</p>
```

```
<p>The name property of the Error object returns the name of the error and the message property returns a description of the error:</p>
```

```
<p id="demo" style="color:red"></p>
```

```
<script>
```

```
try {
```

```
    addalert("Welcome guest!");
```

```
}
```

```
catch(err) {
```

```
document.getElementById("demo").innerHTML = err.message;
```

```
}
```

```
</script>
```

```
</body>
```

JavaScript Variables

Variables are Containers for Storing Data

JavaScript Variables can be declared in 4 ways:

- Automatically

They are automatically declared when first used:

```
x = 5;  
y = 6;  
z = x + y;
```

- Using **var**

```
var x = 5;  
var y = 6;  
var z = x + y;
```

- Using **let**

```
let x = 5;  
let y = 6;  
let z = x + y;
```

- Using **const**

```
const x = 5;  
const y = 6;  
const z = x + y;
```

When to Use var, let, or const?

1. Always declare variables
2. Always use const if the value should not be changed
3. Always use const if the type should not be changed (Arrays and Objects)
4. Only use let if you can't use const
5. Only use var if you MUST support old browsers.

Example 1

```
<html>
```

```
<body>

<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are undeclared.</p>

<p>They are automatically declared when first used.</p>

<p id="demo"></p>

<script>

x = 5;

y = 6;

z = x + y;

document.getElementById("demo").innerHTML = "The value of z is: " + z;

</script>

</body>

</html>
```

Example2

```
<html>

<body>

<h1>JavaScript Variables</h1>

<p>Create a variable, assign a value to it, and display it:</p>

<p id="demo"></p>

<script>

let carName = "Volvo";

document.getElementById("demo").innerHTML = carName;

</script>

</body>
```

</html>

One Statement, Many Variables

```
let person = "John Doe", carName = "Volvo", price = 200;
```

JavaScript Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation s
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

JavaScript Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type

>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Example3

```
<html>
<body>
<h1>JavaScript String Operators</h1>
<h2>The += Operator</h2>
<p>The assignment operator += can concatenate strings.</p>
<p id="demo"></p>
<script>
let text1 = "What a very ";
text1 += "nice day";
document.getElementById("demo").innerHTML = text1;
</script>
</body> </html>
```

Example4

```
<html>
```

```
<body>

<h1>JavaScript String Operators</h1>

<h2>The += Operator</h2>

<p>The assignment operator += can concatenate strings input from text1 and
text2.</p>

<p id="demo"></p>

<input type="text" id="t1" >

<input type="text" id="t2" >

<button type="button" onclick="myFunction()">Test Input</button>

<script>

function myFunction() {

    let x1 = document.getElementById("t1").value;

    let x2 = document.getElementById("t2").value;

    x1 += x2;

    document.getElementById("demo").innerHTML = x1;

}

</script>

</body> </html>
```

JavaScript Logical Operators

Operator	Description
&&	logical and
	logical or

! logical not

JavaScript has 8 Data types

String
Number
Bigint
Boolean
Undefined
Null
Symbol
Object

JavaScript Arrays

An array is a special variable, which can hold more than one value:

```
const cars = ["Saab", "Volvo", "BMW"];
```

Example

```
<html>

<body>

<h1>JavaScript Arrays</h1>

<p id="demo"></p>

<script>

const cars = ["Ford", "Volvo", "BMW"];

document.getElementById("demo").innerHTML = cars;

</script>

</body>
```

</html>

JavaScript Objects

JavaScript object is a variable that can store multiple data in key-value pairs.

Example

// student object

<script>

```
const student = {
```

```
    firstName: "Jack",
```

```
    rollNo: 32,
```

```
};
```

```
student.rollNo=35;
```

```
console.log(student);
```

//Delete object property

```
delete student.rollNo;
```

```
console.log(student);
```

</script>

JavaScript for Loop

Loops are handy, if you want to run the same code over and over again, each time with a different value.

Often this is the case when working with arrays:

Example

<html>

<body>

```
<h2>JavaScript For Loop</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const cars = ["BMW", "Volvo", "Saab", "Ford", "Fiat", "Audi"];
```

```
let text = "";
```

```
for (let i = 0; i < cars.length; i++) {
```

```
    text += cars[i] + "<br>";
```

```
}
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body>
```

```
</html>
```

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions.

You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

JavaScript if, else, and else if

- Use **if** to specify a block of code to be executed, if a specified condition is true
- Use **else** to specify a block of code to be executed, if the same condition is false
- Use **else if** to specify a new condition to test, if the first condition is false
- Use **switch** to specify many alternative blocks of code to be executed

```
if (condition) {
```

```
// block of code to be executed if the condition is true
} else {
    // block of code to be executed if the condition is false
}
```

Example

```
<html>
<body>
<h2>JavaScript Math.random()</h2>
<p id="demo"></p>
<script>
let text;
if (Math.random() < 0.5) {
    text = "<a href='https://w3schools.com'>Visit W3Schools</a>";
} else {
    text = "<a href='https://wwf.org'>Visit WWF</a>";
}
document.getElementById("demo").innerHTML = text;
</script>
</body>
</html>
```


JavaScript Function and Function Expressions

A function is an independent block of code that performs a specific task, while a function expression is a way to store functions in variables.

JavaScript Function Syntax

A JavaScript function is defined with the function keyword, followed by a **name**, followed by parentheses ().

Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).

The parentheses may include parameter names separated by commas:
(*parameter1*, *parameter2*, ...)

Example

```
<html>

<body>

<h1>JavaScript Functions</h1>

<p>Call a function which performs a calculation and returns the result:</p>

<p id="demo"></p>

<script>

function myFunction(p1, p2) {

    return p1 * p2;

}

let result = myFunction(4, 3);

document.getElementById("demo").innerHTML = result;

</script></body></html>
```

Note

Function **parameters** are listed inside the parentheses () in the function definition.

Function **arguments** are the **values** received by the function when it is invoked.

Inside the function, the arguments (the parameters) behave as local variables.

Function Return

When JavaScript reaches a return statement, the function will stop executing. If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement. Functions often compute a **return value**. The return value is "returned" back to the "caller":

Example

```
<html>

<body>

<h1>JavaScript Functions</h1>

<p>Call a function which performs a calculation and returns the result:</p>

<p id="demo"></p>

<script>

let x = myFunction(4, 3);

document.getElementById("demo").innerHTML = x;

function myFunction(a, b) {

    return a * b;

}

</script></body></html>
```

Functions Used as Variable Values

Functions can be used the same way as you use variables, in all types of formulas, assignments, and calculations.

Example

```
<html>

<body>

<h1>JavaScript Functions</h1>

<p>Using a function as a variable:</p>

<p id="demo"></p>

<script>

let text = "The temperature is " + toCelsius(77) + " Celsius.";
```

```
document.getElementById("demo").innerHTML = text;

function toCelsius(fahrenheit) {

    return (5/9) * (fahrenheit-32);

}

</script>

</body>

</html>
```

Example

```
<html>

<head> <script>

function myFunction(x,y) {

    let z=x*y;

    document.getElementById("demo").innerHTML =z;

}

</script> </head>

<body>

<h1>HTML DOM Events</h1>

<h2>The onclick Event</h2>

<p>The onclick event triggers a function when an element is clicked on.</p>

<p>Click to trigger a function that will output the result of two arguments passing from two
Input Numbers:</p>

<input type="number" id="n" name="n">

<input type="number" id="n1" name="n1">

<button onclick="myFunction(n.value,n1.value)">Click me</button>

<p id="demo"></p>

</body></html>
```

JavaScript HTML DOM EventListener

The `addEventListener()` method attaches an event handler to the specified element.

The `addEventListener()` method attaches an event handler to an element without overwriting existing event handlers. You can add many event handlers to one element. You can add many event handlers of the same type to one element, i.e two "click" events. You can add event listeners to any DOM object not only HTML elements. i.e the window object.

The `addEventListener()` method makes it easier to control how the event reacts to bubbling. When using the `addEventListener()` method, the JavaScript is separated from the HTML markup, for better readability and allows you to add event listeners even when you do not control the HTML markup. You can easily remove an event listener by using the `removeEventListener()` method.

Syntax

`element.addEventListener(event, function, useCapture);`

- The first parameter is the type of the event (like "click" or "mousedown" or any other [HTML DOM Event](#).)
- The second parameter is the function we want to call when the event occurs.
- The third parameter is a boolean value specifying whether to use event bubbling or event capturing. This parameter is optional.

Example

```
<html>

<body>

<h2>JavaScript addEventListener()</h2>

<p>This example uses the addEventListener() method to attach a click event to a button.</p>

<button id="myBtn">Try it</button>

<p id="demo"></p>

<script>

document.getElementById("myBtn").addEventListener("click", displayDate);

function displayDate() {

    document.getElementById("demo").innerHTML = Date();

}
```

```
</script></body></html>
```

Example

```
<html>
```

```
<body>
```

```
<h2>JavaScript addEventListener()</h2>
```

```
<p>This example uses the addEventListener() method on the window object.</p>
```

```
<p>Try resizing this browser window to trigger the "resize" event handler.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
window.addEventListener("resize", function(){
```

```
    document.getElementById("demo").innerHTML = Math.random();
```

```
});
```

```
</script></body></html>
```

Event	Occurs When	Belongs To
abort	The loading of a media is aborted	UiEvent , Event
blur	An element loses focus	FocusEvent
canplay	The browser can start playing a media (has buffered enough to begin)	Event
change	The content of a form element has changed	Event
click	An element is clicked on	MouseEvent
dblclick	An element is double-clicked	MouseEvent
drag	An element is being dragged	DragEvent

drop	A dragged element is dropped on the target	DragEvent
fullscreenerror	An element can not be displayed in fullscreen mode	Event
input	An element gets user input	InputEvent , Event
keydown	A key is down	KeyboardEvent
keypress	A key is pressed	KeyboardEvent
keyup	A key is released	KeyboardEvent
load	An object has loaded	UiEvent , Event
mousedown	The mouse button is pressed over an element	MouseEvent
mouseenter	The pointer is moved onto an element	MouseEvent
mouseleave	The pointer is moved out of an element	MouseEvent
mousemove	The pointer is moved over an element	MouseEvent
mouseover	The pointer is moved onto an element	MouseEvent
mouseout	The pointer is moved out of an element	MouseEvent
mouseup	A user releases a mouse button over an element	MouseEvent
Mousewheel	Deprecated. Use the wheel event instead	WheelEvent
offline	The browser starts working offline	Event
online	The browser starts working online	Event

The removeEventListener() method

The removeEventListener() method removes event handlers that have been attached with the addEventListener() method:

Example

```
<html>

<head>

<style>

#myDIV {

    background-color: coral;

    border: 1px solid;

    padding: 50px;

    color: white;

    font-size: 20px;

}

</style>

</head>

<body><h2>JavaScript removeEventListener()</h2>

<div id="myDIV">

    <p>This div element has an onmousemove event handler that displays a random number every time you move your mouse inside this orange field.</p>

    <p>Click the button to remove the div's event handler.</p>

    <button onclick="removeHandler()" id="myBtn">Remove</button>

</div>

<p id="demo"></p>

<script>

document.getElementById("myDIV").addEventListener("mousemove", myFunction);
```

```
function myFunction() {  
    document.getElementById("demo").innerHTML = Math.random();  
}  
function removeHandler() {  
    document.getElementById("myDIV").removeEventListener("mousemove", myFunction);  
}  
</script></body></html>
```

HTML DOM Document

getElementsByClassName()

The `getElementsByClassName()` method returns a collection of elements with a specified class name(s).

The `getElementsByClassName()` method returns an [HTMLCollection](#).

HTMLCollection

An HTMLCollection is an array-like collection (list) of HTML elements.

The length Property returns the number of elements in the collection.

The elements can be accessed by index (starts at 0).

An HTMLCollection is live. It is automatically updated when the document is changed.

```
<html>
  <head>
    <style>
      .example {
        border: 1px solid black;
        padding: 8px;
      }
    </style>
  </head>
  <body>
    <h1>The Document Object</h1>
    <h2>The getElementsByClassName() Method</h2>
    <p> Change the background color of all elements with
      class="example":</p>
    <div class="example">A div with class="example"</div>
    <br>
    <div class="example"> A div with class="example" </div>
    <p class="example"> A p element with class="example". </p>
    <p>A <span class="example">span</span> element with
      class="example".</p>
    <script>
      const collection = document.getElementsByClassName("example");
      for (let i = 0; i < collection.length; i++) {
        collection[i].style.backgroundColor = "red";
      }
    </script>
  </body>
</html>
```

Example 2

```
<html>
<head>
  <style>
    .example {
      border: 1px solid black;
      padding: 8px;
    }
  </style>
</head>
<body>

  <h1>The Document Object</h1>
  <h2>The getElementsByClassName() Method</h2>

  <p>Change the background color of all elements with
  class="example":</p>

  <div class="example"> A div with class="example" </div>
  <br>
  <div class="example"> A div with class="example" </div>

  <p class="example"> A p element with class="example". </p>

  <p> A <span class="example"> span </span> element with
  class="example".</p>
  <input type="text" id="t">
  <button type="button" onclick="myFunction(t.value)">change color
  </button>

  <script>
    function myFunction(tt) {
      const collection = document.getElementsByClassName("example");
      for (let i = 0; i < collection.length; i++) {
        collection[i].style.backgroundColor = tt;
      }
    }
  </script>
</body>
</html>
```

Example 3

```
<html>
  <body>
    <h1>The Document Object</h1>
    <h2>The getElementsByName() Method</h2>
    <p>Change the text of the first element with class="example":</p>
    <div class="example">Element1</div>
    <div class="example">Element2</div>
    <script>
      const collection = document.getElementsByClassName("example");
      collection[0].innerHTML = "Hello World!";
    </script>
  </body>
</html>
```

JavaScript Cookies

What are Cookies?

Cookies are data, stored in small text files, on your computer.

When a web server has sent a web page to a browser, the connection is shut down, and the server forgets everything about the user.

Cookies were invented to solve the problem "how to remember information about the user":

- When a user visits a web page, his/her name can be stored in a cookie.
- Next time the user visits the page, the cookie "remembers" his/her name.

Cookies are saved in name-value pairs like:

```
username = John Doe
```

When a browser requests a web page from a server, cookies belonging to the page are added to the request. This way the server gets the necessary data to "remember" information about users.

Create a Cookie with JavaScript

JavaScript can create, read, and delete cookies with the **document.cookie** property.

With JavaScript, a cookie can be created like this:

```
document.cookie = "username=John Doe";
```

You can also add an expiry date (in UTC time). By default, the cookie is deleted when the browser is closed:

```
document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013 12:00:00 UTC";
```

With a path parameter, you can tell the browser what path the cookie belongs to. By default, the cookie belongs to the current page.

```
document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013 12:00:00 UTC; path="/;
```

Read a Cookie with JavaScript

With JavaScript, cookies can be read like this:

```
let x = document.cookie;
```

Change a Cookie with JavaScript

With JavaScript, you can change a cookie the same way as you create it:

```
document.cookie = "username=John Smith; expires=Thu, 18 Dec 2013 12:00:00 UTC; path="/;
```

The old cookie is overwritten.

Delete a Cookie with JavaScript

Deleting a cookie is very simple.

You don't have to specify a cookie value when you delete a cookie.

Just set the expires parameter to a past date:

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path="/;
```

You should define the cookie path to ensure that you delete the right cookie.

Some browsers will not let you delete a cookie if you don't specify the path.

JavaScript Cookie Example

In the example to follow, we will create a cookie that stores the name of a visitor.

The first time a visitor arrives to the web page, he/she will be asked to fill in his/her name. The name is then stored in a cookie.

The next time the visitor arrives at the same page, he/she will get a welcome message.

For the example we will create 3 JavaScript functions:

1. A function to set a cookie value
2. A function to get a cookie value
3. A function to check a cookie value

Example 4

```
<html>
<script>
    function setCookies(cname,cvalue,ExpDay) {
        const d=new Date();
        d.setTime(d.getTime()+ (ExpDay*24*60*60*1000));
        document.cookie= cname
        + "=" + cvalue + "; expires=" + d.toDateString() + "; path=/"
    }

    function getCookie(cname) {
        const cookies=document.cookie.split("; ");
        //document.getElementById("cc").innerHTML=cookies;

        for (let i=0;i<cookies.length;i++){
            //document.getElementById("cc").innerHTML=cookies[i];
            if (cookies[i].startsWith(cname + "="))
                return cookies[i].split("=")[1];
        }
        return "";
    }

    function checkCookie() {
        let user=getCookie("Uname");
        if ( user != "") {
            alert("welcome " + user);
        }
        else
        {
            user= prompt("enter user name", "");
            if (user != "" && user != null)
                {setCookies("Uname",user,30);}
        }
    }
</script>
<body onload="checkCookie()">
<p id="cc"></p>
</body>
</html>
```

What is Bootstrap?

- Bootstrap is a free front-end framework for faster and easier web development
- Bootstrap includes HTML and CSS based design templates for typography, forms, buttons, tables, navigation, modals, image carousels and many other, as well as optional JavaScript plugins
- Bootstrap also gives you the ability to easily create responsive designs

What is Responsive Web Design?

Responsive web design is about creating web sites which automatically adjust themselves to look good on all devices, from small phones to large desktops.

Bootstrap History

Bootstrap was developed by Mark Otto and Jacob Thornton at Twitter, and released as an open source product in August 2011 on GitHub.

In June 2014 Bootstrap was the No.1 project on GitHub!

Bootstrap Versions

This tutorial follows **Bootstrap 3**, which was released in 2013. However, we also cover newer versions; [Bootstrap 4 \(released 2018\)](#) and [Bootstrap 5 \(released 2021\)](#).

[Bootstrap 5](#) is the newest version of [Bootstrap](#); with new components, faster stylesheets, more responsiveness etc. It supports the latest, stable releases of all major browsers and platforms. However, Internet Explorer 11 and down is not supported.

The main differences between Bootstrap 5 and Bootstrap 3 & 4, is that Bootstrap 5 has switched to [JavaScript](#) instead of [jQuery](#).

Note: [Bootstrap 3](#) and Bootstrap 4 is still supported by the team for critical bugfixes and documentation changes, and it is perfectly safe to continue to use them. However, new features will NOT be added to them.

Where to Get Bootstrap?

There are two ways to start using Bootstrap on your own web site.

You can:

- Download Bootstrap from getbootstrap.com
- Include Bootstrap from a CDN

Bootstrap Buttons

Button Styles

Bootstrap provides different styles of buttons:

Basic Default Primary Success Info Warning Danger Link

To achieve the button styles above, Bootstrap has the following classes:

- `.btn`
- `.btn-default`
- `.btn-primary`
- `.btn-success`
- `.btn-info`
- `.btn-warning`
- `.btn-danger`
- `.btn-link`

The following example shows the code for the different button styles:



Example

```
<html lang="en">

<head>

  <title>Bootstrap Example</title>

  <meta charset="utf-8">

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>

</head>

<body>

<div class="container">
```

```
<h2>Button Styles</h2>
```

```
<button type="button" class="btn">Basic</button>
```

```
<button type="button" class="btn btn-default">Default</button>
```

```
<button type="button" class="btn btn-primary">Primary</button>
```

```
<button type="button" class="btn btn-success">Success</button>
```

```
<button type="button" class="btn btn-info">Info</button>
```

```
<button type="button" class="btn btn-warning">Warning</button>
```

```
<button type="button" class="btn btn-danger">Danger</button>
```

```
<button type="button" class="btn btn-link">Link</button>
```

```
</div></body></html>
```

Bootstrap Image Shapes

Rounded Corners:



Circle:



Thumbnail:



Rounded Corners

The `.img-rounded` class adds rounded corners to an image

Circle

The `.img-circle` class shapes the image to a circle

Example

```
<html lang="en">
```

```
<head>
```

```
<title>Bootstrap Example</title>
```

```
<meta charset="utf-8">

<meta name="viewport" content="width=device-width, initial-scale=1">

<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>

</head>

<body>

<div class="container">

  <h2>Rounded Corners</h2>

  <p>The .img-rounded class adds rounded corners to an image (not available in IE8):</p>

</div></body></html>
```

Bootstrap Tables

Bootstrap Basic Table

A basic Bootstrap table has a light padding and only horizontal dividers.

The .table class adds basic styling to a table:

Example

```
<html lang="en">

<head>

  <title>Bootstrap Example</title>

  <meta charset="utf-8">

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<h2>Basic Table</h2>
```

```
<p>The .table class adds basic styling (light padding and only horizontal dividers) to a table:</p>
```

```
<table class="table">
```

```
<thead>
```

```
<tr>
```

```
<th>Firstname</th>
```

```
<th>Lastname</th>
```

```
<th>Email</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
<tr>
```

```
<td>John</td>
```

```
<td>Doe</td>
```

```
<td>john@example.com</td>
```

```
</tr> <tr>
```

```
<td>Mary</td>
```

```
<td>Moe</td>
```

```
<td>mary@example.com</td>
```

```
</tr>
```

```
<tr>
```

```
<td>July</td>
```

```
<td>Dooley</td>
```

```
<td>july@example.com</td>
```

```
</tr>

</tbody>

</table>

</div>

</body>

</html>
```

Bootstrap Dropdowns

A dropdown menu is a toggleable menu that allows the user to choose one value from a predefined list:

Example

```
<html>

<head>

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>

</head>

<body>

<div class="container">

  <h2>Dropdowns</h2>

  <p>The .dropdown class is used to indicate a dropdown menu.</p>

  <p>Use the .dropdown-menu class to actually build the dropdown menu.</p>

  <p>To open the dropdown menu, use a button or a link with a class of .dropdown-toggle and data-
toggle="dropdown".</p>

  <div class="dropdown">

    <button class="btn btn-primary dropdown-toggle" type="button" data-
toggle="dropdown">Dropdown Example
```

```

<span class="caret"></span></button>

<ul class="dropdown-menu">

  <li><a href="#">HTML</a></li>

  <li><a href="#">CSS</a></li>

  <li><a href="#">JavaScript</a></li>

</ul>

</div>

</div>

</body>

</html>

```

Bootstrap Navigation Bar

Navigation Bars

A navigation bar is a navigation header that is placed at the top of the page:

With Bootstrap, a navigation bar can extend or collapse, depending on the screen size.

A standard navigation bar is created with `<nav class="navbar navbar-default">`.

The following example shows how to add a navigation bar to the top of the page:

Example

```

<html lang="en">

<head>

  <title>Bootstrap Example</title>

  <meta charset="utf-8">

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>

```

```
</head>

<body>

<nav class="navbar navbar-default">

  <div class="container-fluid">

    <div class="navbar-header">

      <a class="navbar-brand" href="#">WebSiteName</a>

    </div>

    <ul class="nav navbar-nav">

      <li class="active"><a href="#">Home</a></li>

      <li><a href="#">Page 1</a></li>

      <li><a href="#">Page 2</a></li>

      <li><a href="#">Page 3</a></li>

    </ul>

  </div>

</nav>

<div class="container">

  <h3>Basic Navbar Example</h3>

  <p>A navigation bar is a navigation header that is placed at the top of the page.</p>

</div>

</body>

</html>
```